

DBMS Project Part 2. Storage Layer

Goal

Implement a minimal but real storage engine that can:

- store tables on disk in a defined physical format,
- insert/read/update/delete records,
- Persist across restarts,
- maintain schema metadata consistently,
- support at least one concrete storage organization strategy (heap/B+Tree/LSM/hash, etc.).

Supported directions (choose one)

You must pick one of two tracks. Both tracks must expose the same logical API (insert/get/update/delete/scan), but the physical design differs.

Track A. OLTP Row-Store (page-based)

Classic slotted pages / heap files, optimized for point reads + updates.

Track B. OLAP Column-Store (Column/hybrid-style)

Append-optimized, columnar layout for scan-heavy analytics. Parquet is explicitly allowed as a target format. You can also try using other existing formats (Feather, ORC, Arrow, etc.) or implement one on your own.

Common API contract

Your storage module must provide (names are illustrative; adapt to your language and project):

- `create_table(table_name, schema)`
- `drop_table(table_name)` (optional but recommended)
- `open_table(table_name) » TableHandle`
- `insert(table, record) » RID`
- `get(table, rid) » record | null`
- `update(table, rid, new_record) » bool`
- `delete(table, rid) » bool`
- `scan(table) » iterator/stream of (RID, record)` (needed soon for sequential scans)

Data model requirements

You must support:

- fixed-length types (e.g., INT/FLOAT)
- variable-length types (e.g., VARCHAR/TEXT)
- Optionally, anything else by your choice

Requirements

Basic Requirements

- Block/page structure implemented (fixed page size)
- Page header + payload layout defined and documented.

Basic record I/O

- write/insert, read/get, update, delete work correctly.
- Fixed + variable-length records
- Must handle variable-length safely (length prefixes, offsets, or indirection).

Schema/metadata storage + consistency

- Persist schema; on restart, tables can be reopened with correct types/columns.

Storage organization strategy (pick one)

- Heap files (free-space mgmt + page directory)
- Hash-based storage
- B+Tree-backed table or index
- LSM-like append + compaction
- Self-implemented index over Parquets
- or another justified approach

Advanced Requirements (optional, but recommended to try at least one)

- Compression feature (OLAP-friendly): dictionary / delta / RLE / bit-packing on at least one column type (even if simplistic). Parquet-style encodings are explicitly relevant here.
- Zone maps / min-max stats per segment/page for scan skipping (OLAP track strongly encouraged).
- PAX page layout (hybrid): page-level row grouping but within-page column grouping, bridging OLTP/OLAP.
- Crash-safety-lite: atomic file replace for metadata/manifest, checksums, or write-ahead log skeleton (not full transactions yet).
- Benchmarks: a small micro-benchmark suite demonstrating at least one meaningful tradeoff.

Track A details. Row-store (OLTP)

This is the canonical approach if you want straightforward update/delete semantics.

Recommended physical design

- Slotted pages (slot directory + free space) for variable-length records.
- Variable-length fields:
 - inline if small,
 - otherwise overflow/indirection (overflow pages or blob store).

Record identity

RID = (page_id, slot_id) (common), with a tombstone/redirect pattern if you allow record movement.

What “update” mean

In-place update if the new record fits.

If it doesn't fit: either

- move record + write forwarding pointer (tombstone), or
- allocate overflow space and keep an indirection.

Storage organization strategy ideas

- HeapFile + Free Space Map (simplest production-style baseline).
- B+Tree index on a chosen key (even if only unique integer keys).
- Hash directory (extendible hash optional).

Track B details. Column-store (OLAP) with Parquet option

Two acceptable implementations

Option B1: Columnar Storage

Physical layout (DSM core)

- Each table is decomposed into per-column storage (separate files or separate regions/segments inside a table file).
- Each stored value is associated with a Record ID (RID) / position so that rows can be reconstructed (late materialization / tuple reconstruction conceptually).
- Explicitly document your RID scheme (dense 0..N-1, or stable RID with free-list, etc.).

Optimization Options

- Compression

If you implement compression, use at least one technique discussed in the course such as RLE, Dictionary, Delta, Bit-packing.

You must document: encoding choice, decode cost, and how it integrates with scans.

Option B2: Real Parquet via library integration

You may use Arrow/Parquet libraries to write/read Parquet files, but you must still:

- design the table-to-files layout,
- implement your own catalog/metadata and RID mapping,
- implement update/delete semantics via a delta layer (below),
- expose the same storage API.

Handling update/delete in an OLAP-friendly way

Parquet/immutable segments make in-place updates painful. A realistic approach:

- Base segments are immutable (row groups / segment files).
- updates: append a new version keyed by RID or primary key,
- deletes: maintain a delete bitmap or “position delete” list per segment.
- Reads (get and scan) must merge base + delta logically.

This mirrors real OLAP systems where write-optimized deltas are reconciled later (compaction/merge).

Optimization Options

- Projection-aware reads: if query only needs 2 columns, avoid reading others.
- Statistics per row group (min/max) for skipping. Parquet-like systems use metadata for this style of pruning.
- Encoding/compression: dictionary/RLE/delta/bit-pack, even simplified.

Deliverables

1. Code in your repo: storage module + CLI or test harness.
2. README / design doc including:
 - a. chosen track + rationale (workload assumptions),
 - b. on-disk layout description (diagrams encouraged),
3. RID semantics,
4. How restart/recovery is handled (even if minimal)
5. Complexity notes (insert/update/delete/scan)
6. Tests:
 - a. correctness tests for CRUD + restart,
 - b. edge cases: varlen boundaries, page-full behavior, delete tombstones, etc.
7. Demo script (commands or automated run) proving persistence + API correctness.